

Yes, we can absolutely lock this in. To guarantee that sheets 15 through 26 are permanently walled off as a "Protected Tail Block" at the end of your workbook, we need to implement a Tail Block Shield within the framework's sorting logic.

By updating two specific functions, you guarantee two things:

1. Strict Ranking: Any unknown sheet, newly generated output, or manually created user tab automatically receives a rank of **145**, wedging it perfectly between **Archive - Demo P** (140) and **Framework Timing Report** (150).
2. Insertion Cap: Whenever the framework builds a fast canvas, the insertion algorithm checks the "Tail Block Shield." If the new sheet is not a system/template sheet, its maximum allowed placement is physically capped to stop directly before the **Framework Timing Report**.

1. Refactor the Global Ranking Dictionary

Replace your existing `getGlobalSheetSortRankByName_` function with this updated version. This perfectly mirrors your requested 1-26 layout.

JavaScript

None

```
function getGlobalSheetSortRankByName_(sheetName) {  
    sheetName = String(sheetName || "").trim();  
  
    // 1-14: Active & Import Workspace  
  
    if (sheetName === INDEX_SHEET) return 10;  
  
    if (sheetName === SHEET_TYPE.DEMO_P ||  
sheetName.indexOf(DEMO_P_PREFIX) === 0) return 20;  
  
    if (sheetName.indexOf(MASTER_LIST_PREFIX) === 0) return 30;  
  
    if (sheetName.indexOf(MONTHLY_CHANGE_REPORT_PREFIX) === 0) return  
40;  
}
```

```
    if (sheetName === DISENROLLED_EXCLUSION_SHEET ||  
/^Disenrolled\b/i.test(sheetName) && sheetName.indexOf("Template")  
=== -1) return 50;  
  
    if (sheetName.indexOf("Raw Data") === 0) return 60;  
  
    if (sheetName.indexOf(BANNER_PREFIX) === 0 ||  
sheetName.indexOf(BANNER_REPORT_ALT_PREFIX) === 0) return 70;  
  
    if (sheetName.indexOf(CARE_PLAN_DUE_PREFIX) === 0 ||  
sheetName.indexOf(CARE_PLAN_DUE_DATE_ALT_PREFIX) === 0) return 80;  
  
    if (sheetName.indexOf(UNLOCKED_PREFIX) === 0 ||  
sheetName.indexOf("Unlocked Care Plan") === 0) return 90;  
  
    if (/^B(\b|\s)/i.test(sheetName)) return 100;  
  
    if (/^CD(\b|\s)/i.test(sheetName)) return 110;  
  
    if (/^UC(\b|\s)/i.test(sheetName)) return 120;  
  
    if (/^RD(\b|\s)/i.test(sheetName)) return 130;  
  
    if (sheetName === DEMO_P_ARCHIVE_SHEET) return 140;  
  
  
    // 15-26: The Protected Tail Block  
  
    if (sheetName === RFF_TIMING_SHEET || sheetName === "Framework  
Timing Report") return 150;  
  
    if (sheetName === RFF_TEST_DASHBOARD_SHEET || sheetName ===  
"Dashboard Quality Report") return 160;  
  
    if (sheetName === RFF_DASHBOARD_SHEET) return 170;  
  
    if (sheetName === "Template - Banner Report") return 180;  
  
    if (sheetName === "Template - Care Plan Due") return 190;
```

```

    if (sheetName === "Template - Unlocked Care Plan") return 200;
    if (sheetName === "Template - Raw Data") return 210;
    if (sheetName === "Template - Demo P") return 220;
    if (sheetName === "Template - Disenrolled Exclusion") return 230;
    if (sheetName === "Template - Master List") return 240;
    if (sheetName === "Template - Monthly Change") return 250;
    if (sheetName === RFF_BASE_TEMPLATE_NAME) return 260;

    // UNKNOWN / NEW SHEETS

    // Any new sheet created manually or by output workflows gets
    rank 145,

    // anchoring it immediately BEFORE the Framework Timing Report
    (150).

    return 145;
}

```

2. Implement the Insertion Tail Block Shield

Replace your existing `getGovernedSheetInsertIndex_` function. This prevents new programmatic sheets from skipping the queue and landing at the very end of the workbook.

JavaScript

None

```

function getGovernedSheetInsertIndex_(ss, sheetName) {

    ss = ss || SpreadsheetApp.getActiveSpreadsheet();

    const targetName = String(sheetName || "").trim();

```

```
const targetRank = getGlobalSheetSortRankByName_(targetName);

// --- THE TAIL BLOCK SHIELD ---

// If the sheet being inserted is an operational or unknown sheet
(Rank < 150),

// it must NEVER be placed inside or after the protected tail
block.

let maxAllowedIndex = ss.getNumSheets() + 1;

if (targetRank < 150) {

    const timingSheet = ss.getSheetByName("Framework Timing
Report") || ss.getSheetByName(RFF_TIMING_SHEET);

    // If the Timing Report exists, the max index is exactly its
current index (shifting it to the left)

    if (timingSheet) maxAllowedIndex = timingSheet.getIndex();

}

let targetIndex = 1;

ss.getSheets().forEach(function(sheet) {

    const name = String(sheet.getName() || "").trim();

    if (!name || name === targetName) return;

    const rank = getGlobalSheetSortRankByName_(name);

    if (rank < targetRank) {
```

```

        targetIndex = Math.max(targetIndex, sheet.getIndex() + 1);
    }

    if (rank === targetRank &&
compareSheetNamesByGlobalOrder_(name, targetName) < 0) {

        targetIndex = Math.max(targetIndex, sheet.getIndex() + 1);
    }

});

return Math.max(1, Math.min(targetIndex, maxAllowedIndex));
}

```

The great news is that you actually already locked them into the 1st and 2nd positions with the code we just implemented!

Because of how the `getGlobalSheetSortRankByName_` function is structured, the mathematical ranking explicitly guarantees their spot:

- **Index** is assigned Rank **10** (the lowest possible number).
- **Demo P** is assigned Rank **20** (the second lowest number).
- The next highest sheet is **Master List** at Rank **30**.

When the `enforceGlobalSheetSortOrder_` engine runs, it sorts the tabs strictly by these numbers. Because **10** will always be smaller than **20**, and **20** will always be smaller than **30**, Index will always be Position 1, and Demo P will always be Position 2.

The Bulletproof "Front Block Shield"

If you want to be absolutely, 100% certain that the framework's sheet-*insertion* engine never accidentally drops a brand new sheet into position 1 or 2 (even for a split second before the sort runs), we can add a "Front Block Shield" to the `getGovernedSheetInsertIndex_` function, right next to our Tail Block Shield.

Here is the final, ironclad version of that function:

JavaScript

None

```
function getGovernedSheetInsertIndex_(ss, sheetName) {

  ss = ss || SpreadsheetApp.getActiveSpreadsheet();

  const targetName = String(sheetName || "").trim();

  const targetRank = getGlobalSheetSortRankByName_(targetName);

  // --- THE TAIL BLOCK SHIELD ---

  let maxAllowedIndex = ss.getNumSheets() + 1;

  if (targetRank < 150) {

    const timingSheet = ss.getSheetByName("Framework Timing Report") || ss.getSheetByName(RFF_TIMING_SHEET);

    if (timingSheet) maxAllowedIndex = timingSheet.getIndex();

  }

  // --- THE FRONT BLOCK SHIELD ---

  // If the sheet being inserted is NOT Index (10) or Demo P (20),
  // its absolute minimum starting position is forced to 3.

  let targetIndex = (targetRank > 20) ? 3 : 1;

  ss.getSheets().forEach(function(sheet) {

    const name = String(sheet.getName() || "").trim();

    if (!name || name === targetName) return;
```

```

    const rank = getGlobalSheetSortRankByName_(name);

    if (rank < targetRank) {

        targetIndex = Math.max(targetIndex, sheet.getIndex() + 1);

    }

    if (rank === targetRank &&
compareSheetNamesByGlobalOrder_(name, targetName) < 0) {

        targetIndex = Math.max(targetIndex, sheet.getIndex() + 1);

    }

});

return Math.max(1, Math.min(targetIndex, maxAllowedIndex));

}

```

With this in place, your workbook is mathematically sealed at both ends: **Index** and **Demo P** are permanently locked at the front, and your system sheets are permanently locked at the back.

The 17.283-second bottleneck in the Dashboard Quality Workflow is almost entirely caused by the **styleDashboardQualityUpdatedSections_** function. When the script writes the H-N batch, it constructs massive 2D arrays containing individual font weights, backgrounds, and sizes for *every single cell* in that block, and pushes them to the sheet row-by-row.

To execute Codex's recommendation ("Skip write when generated payload signature is unchanged"), we will implement a Payload Signature Skip.

Because your diagnostic data (like the Framework Health Check) rarely changes from run to run, we can hash the JSON array of the audit results. If the hash matches the previous run, we skip the heavy DOM insertion and 2D formatting arrays entirely, and use a lightning-fast **RangeList** to *only* update the "Last Updated" timestamp cells.

Here is the implementation.

1. Refactor `runDashboardQualityFull`

Replace your existing `runDashboardQualityFull()` function with this optimized version:

JavaScript

None

```
function runDashboardQualityFull() {  
    return runFrameworkTimed_("Dashboard Quality Workflow",  
function(timing) {  
        const qualitySheet = ensureDashboardQualityReportSheet_();  
        const previousDefer = RFF_DASHBOARD_QUALITY_DEFER_WRITES_  
        RFF_DASHBOARD_QUALITY_DEFER_WRITES_ = true;  
  
        try {  
            runDashboardQualitySectionIfDue_(RFF_HEALTH_CHECK_SHEET,  
"Section H Framework Health Check", runFrameworkHealthCheck,  
timing);  
  
            runDashboardQualitySectionIfDue_(RFF_PERFORMANCE_SUMMARY_KEY,  
"Section I Performance Summary",  
runDashboardQualityPerformanceSummary_, timing);  
  
            runDashboardQualitySectionIfDue_(RFF_MASTER_LIST_HEALTH_KEY,  
"Section J Master List Validation",  
runDashboardQualityMasterListHealthCheck_, timing);  
  
            runDashboardQualitySectionIfDue_(RFF_CP_SYNC_DIAGNOSTICS_KEY,  
"Section K Care Plan Sync Validation",  
runDashboardQualityCarePlanSyncDiagnostics_, timing);  
  
            runDashboardQualitySectionIfDue_(RFF_WORKFLOW_SYNC_VERIFICATION_KEY
```



```
, "Section L Workflow & Synchronization Verification",
runDashboardQualityWorkflowSyncVerification_, timing);

    } finally {

        RFF_DASHBOARD_QUALITY_DEFER_WRITES_ = previousDefer;

    }

    const summaryRows = [["Metric", "Value", "Status", "Quality
Notes"]];

    buildFrameworkSummaryRows_().forEach(function(row) {

        summaryRows.push(row);

    });

    const signoffRows = [];

    appendCombinedDashboardSignOffRows_(signoffRows);

    while (signoffRows.length && signoffRows[0].length === 1 &&
String(signoffRows[0][0] || "") === "") {

        signoffRows.shift();

    }

    const now = String(new Date().getTime());

    const props = PropertiesService.getDocumentProperties();

    props.setProperty(RFF_QA_SECTION_PROP_PREFIX +
"Summary_LAST_RUN", now);
```

```
    props.setProperty(RFF_QA_SECTION_PROP_PREFIX +
"Signoff_LAST_RUN", now);

    // --- OPTIMIZATION: PAYLOAD SIGNATURE SKIP ---

    const sectionsToProcess = [

        { key: RFF_HEALTH_CHECK_SHEET, title: "SECTION H - FRAMEWORK
HEALTH CHECK", rows:
getDashboardQualitySectionRows_(RFF_HEALTH_CHECK_SHEET) },

        { key: RFF_PERFORMANCE_SUMMARY_KEY, title: "SECTION I -
PERFORMANCE SUMMARY", rows:
getDashboardQualitySectionRows_(RFF_PERFORMANCE_SUMMARY_KEY) },

        { key: RFF_MASTER_LIST_HEALTH_KEY, title: "SECTION J - MASTER
LIST VALIDATION", rows:
getDashboardQualitySectionRows_(RFF_MASTER_LIST_HEALTH_KEY) },

        { key: RFF_CP_SYNC_DIAGNOSTICS_KEY, title: "SECTION K - CARE
PLAN SYNC VALIDATION", rows:
getDashboardQualitySectionRows_(RFF_CP_SYNC_DIAGNOSTICS_KEY) },

        { key: RFF_WORKFLOW_SYNC_VERIFICATION_KEY, title: "SECTION L
- WORKFLOW & SYNCHRONIZATION VERIFICATION", rows:
getDashboardQualitySectionRows_(RFF_WORKFLOW_SYNC_VERIFICATION_KEY)
},

        { key: "Summary", title: "SECTION M - SUMMARY", rows:
summaryRows },

        { key: "Signoff", title: "SECTION N - SIGNOFF", rows:
signoffRows.slice(1) }

    ];
```

```
let requiresFullWrite = false;

const sectionWrites = [];

sectionsToProcess.forEach(function(section) {

    const hashKey = RFF_QA_SECTION_PROP_PREFIX + section.key +
"_DATA_HASH";

    const currentHash =
computeStableHash_(JSON.stringify(section.rows));

    const previousHash = props.getProperty(hashKey);

    // If the data structure changed, trigger a full batch DOM
write and format

    if (currentHash !== previousHash) {

        requiresFullWrite = true;

        props.setProperty(hashKey, currentHash);

    }

    sectionWrites.push({

        title: section.title,

        rows:
buildTimestampedDashboardQualitySectionRows_(section.key,
section.rows || [])

    });
```

```

    });

    if (requiresFullWrite) {

        replaceDashboardQualitySectionsRows_(qualitySheet,
        sectionWrites);

        markFrameworkStep_(timing, "Dashboard Quality Sections H-N
        batch updated (Data Changed)");

    } else {

        // Fast-path: The data is identical to the last run. Only
        update the timestamps.

        updateDashboardQualityTimestampsOnly_(qualitySheet,
        sectionsToProcess.map(function(s) { return s.title; }));

        markFrameworkStep_(timing, "Dashboard Quality Sections H-N
        data unchanged - timestamps updated via fast-path");

    }

    writeCombinedFrameworkTimingReport_();

});

}

```

2. Add the Fast-Path Timestamp Helper

Add this helper function below `runDashboardQualityFull`. This utilizes the `RangeList` API to write the new timestamp string to all H-N sections simultaneously in a single API call, completely bypassing the heavy array formatting engine.

JavaScript

None

```
function updateDashboardQualityTimestampsOnly_(sheet,
sectionTitles) {

    if (!sheet || !sectionTitles || !sectionTitles.length) return;

    const bounds = getDashboardQualitySectionBoundsMap_(sheet);

    const ranges = [];

    const now = new Date();

    sectionTitles.forEach(function(title) {

        if (bounds[title]) {

            // The timestamp is consistently placed 2 rows below the
            section replacement boundary, in column B

            const timestampRow = bounds[title].replaceStartRow + 2;

            ranges.push("B" + timestampRow);

        }

    });

    if (ranges.length > 0) {

        try {

            sheet.getRangeList(ranges).setValue(now).setNumberFormat("mm/dd/yyyy
            hh:mm:ss");

        } catch (err) {
```

```
        logBestEffortWarning_("Timestamp fast-path update failed: " +  
err.message);  
  
    }  
  
}  
  
}
```

You have incredibly sharp instincts. You are exactly right.

The script's built-in safety mechanisms are essentially sabotaging its performance. Because the framework is paranoid about the structural integrity of the Dashboard Quality Report, it triggers what I call a "Domino Rebuild Loop."

Here is exactly what is happening under the hood:

1. The 78x Read Penalty: Before the script writes *any* section, it calls `hasDashboardQualityTemplateShell_()`. To verify the sheet is safe to write to, this function calls `findDashboardQualitySectionRow_` for all 13 sections. Because it reads the sheet individually for each check, updating Sections H–N forces the script to read the exact same column 78 times.
2. The Domino Rebuild: If the script decides to write the sections one by one, and it miscalculates a boundary, it panics and calls `rebuildDashboardQualityReportShellCompact_()`. This completely wipes the sheet, redraws all 13 sections, applies borders to the whole grid, and then tries the next section... which triggers another panic and *another* total rebuild.

We need to optimize the shell verification so it only reads the sheet once, and we need to pass a flag to the batch writer so it stops redundantly checking the shell for every single section.

1. Optimize the Shell Verification

Replace your existing `hasDashboardQualityTemplateShell_` function with this lightning-fast version. Instead of 13 separate `getValues()` calls, it pulls the column once, loads the titles into a JavaScript `Set`, and verifies everything in memory.

JavaScript

None

```
function hasDashboardQualityTemplateShell(sheet) {  
    if (!sheet) return false;  
  
    // 1. Check title block (1 API call)  
  
    const titleValues = sheet.getRange(1, 1, 1,  
3).getDisplayValues()[0].map(function(value) {  
        return String(value || "").trim();  
    });  
  
    if (titleValues[0] !== "Dashboard Quality Report") return false;  
  
    if (titleValues[1] !== "- v" + MASTER_LIST_MERGE_ML_VERSION + "  
-") return false;  
  
    // 2. Read the sheet exactly ONCE (1 API call)  
  
    const lastRow = Math.max(sheet.getLastRow(), 1);  
  
    const values = sheet.getRange(1, 1, lastRow, 1).getValues();  
  
    // 3. Load all existing titles into a high-speed Set  
  
    const existingTitles = new Set();  
  
    for (let i = 0; i < values.length; i++) {  
        const text = String(values[i][0] || "").trim();  
  
        if (text) existingTitles.add(text);  
    }  
}
```

```

// 4. Verify all required sections against the Set (0 API calls)
for (let i = 0; i < RFF_DASHBOARD_QUALITY_SECTIONS.length; i++) {
    if
(!existingTitles.has(RFF_DASHBOARD_QUALITY_SECTIONS[i].title))
return false;
}

return true;
}

```

2. Stop the Redundant Rebuilds

Update your `replaceDashboardQualitySectionRows_` (singular) function to accept a skip flag.

Find this block near the top of the function:

JavaScript

```

None
if (!hasDashboardQualityTemplateShell_(sheet)) {
    rebuildDashboardQualityReportShellCompact_(sheet);
}

```

Replace it with this:

JavaScript

None

```
// Only check and rebuild if we aren't explicitly skipping it
from a batch run

if (!(options && options.skipShellCheck) &&
!hasDashboardQualityTemplateShell_(sheet)) {

    rebuildDashboardQualityReportShellCompact_(sheet);

}
```

3. Pass the Flag from the Batch Writer

Finally, update the fallback loop at the very bottom of your `replaceDashboardQualitySectionsRows_` (plural) function. Add `skipShellCheck: true` to the options payload so the batch writer checks the shell once at the top, and never again.

JavaScript

None

```
sectionWrites.slice().sort(function(a, b) {

    const aBound = bounds[a.title] || { replaceStartRow: 0 };
    const bBound = bounds[b.title] || { replaceStartRow: 0 };

    return bBound.replaceStartRow - aBound.replaceStartRow;

}).forEach(function(sectionWrite) {

    replaceDashboardQualitySectionRows_(sheet, sectionWrite.title,
sectionWrite.rows, {

        sectionBounds: bounds,

        deferStyle: true,

        skipBreakApart: true,

        skipShellCheck: true // <--- Add this flag to stop the Domino
Rebuild
```

```
});  
  
});
```

It is the exact same architectural flaw. Every single time the framework finishes a step and logs it, it calls `appendRuntimeTimingToFrameworkTimingReport_`. That function calls `ensureFrameworkTimingReport_` (which checks the shell), and then it calls `replaceFrameworkTimingSectionRows_` (which checks the shell *again*).

Because the old shell check reads the sheet column row-by-row and verifies headers cell-by-cell for all 4 sections, it's performing dozens of redundant API calls for every single telemetry tick.

Here is the patch to optimize the Framework Timing Report shell verification and stop its domino rebuilds.

1. Optimize the Timing Shell Verification

Replace your existing `hasFrameworkTimingReportShell_` function with this high-speed, single-read `Set` version.

JavaScript

None

```
function hasFrameworkTimingReportShell_(sheet) {  
    if (!sheet) return false;  
  
    // 1. Check title block (1 API call)  
  
    const titleValues = sheet.getRange(1, 1, 1,  
3).getDisplayValues()[0].map(function(value) {  
        return String(value || "").trim();  
    });  
  
    if (titleValues[0] !== "Framework Timing Report") return false;
```

```
    if (titleValues[1] !== "- v" + MASTER_LIST_MERGE_ML_VERSION + "-") return false;

    // 2. Read the sheet exactly ONCE (1 API call)
    const lastRow = Math.max(sheet.getLastRow(), 1);
    const values = sheet.getRange(1, 1, lastRow, 1).getValues();

    // 3. Load all existing titles into a high-speed Set
    const existingTitles = new Set();
    for (let i = 0; i < values.length; i++) {
        const text = String(values[i][0] || "").trim();
        if (text) existingTitles.add(text);
    }

    // 4. Verify all required sections against the Set (0 API calls)
    const sections = getFrameworkTimingSectionRegistry_();
    for (let i = 0; i < sections.length; i++) {
        if (!existingTitles.has(sections[i].title)) return false;
    }

    return true;
}
```

2. Add the Skip Flag to the Timing Writer

Update the very top of `replaceFrameworkTimingSectionRows_` to accept and respect the `skipShellCheck` option.

Find this block:

JavaScript

None

```
function replaceFrameworkTimingSectionRows_(sheet, sectionId, rows,
options) {

    if (!sheet) return;

    const section = getFrameworkTimingSectionForId_(sectionId);

    if (!section) throw new Error("Unknown Framework Timing Report
section: " + sectionId);

    if (!hasFrameworkTimingReportShell_(sheet)) {

        rebuildFrameworkTimingReportShellCompact_(sheet);

    }
}
```

Replace it with this:

JavaScript

None

```
function replaceFrameworkTimingSectionRows_(sheet, sectionId, rows,
options) {

    if (!sheet) return;
```

```

    const section = getFrameworkTimingSectionForId_(sectionId);

    if (!section) throw new Error("Unknown Framework Timing Report
section: " + sectionId);

    // Only check and rebuild if we aren't explicitly skipping it
    from a batch/verified run

    if (!(options && options.skipShellCheck) &&
!hasFrameworkTimingReportShell_(sheet)) {

        rebuildFrameworkTimingReportShellCompact_(sheet);

    }

```

3. Pass the Flag from the Parent Callers

Finally, update the two main functions that write to the Timing Report so they pass the new flag. Because they both run `ensureFrameworkTimingReport_()` at the very top of their execution (which already guarantees the shell is perfect), checking it again during the write phase is a total waste of execution time.

In `writeCombinedFrameworkTimingReport_`, update the `options` object:

JavaScript

None

```

    const recommendationRows =
buildFrameworkTimingRecommendationRows_(issueRows);

    // Add skipShellCheck to stop redundant verifications for
    Sections A, B, and C

    const options = { skipStyle: true, skipShellCheck: true };

```

```

    replaceFrameworkTimingSectionRows_(sheet, "A", processRows,
options);

    replaceFrameworkTimingSectionRows_(sheet, "B", issueRows,
options);

    replaceFrameworkTimingSectionRows_(sheet, "C",
recommendationRows, options);

```

In `appendRuntimeTimingToFrameworkTimingReport_`, update the `options` parameter in the `replace...` call:

JavaScript

```

None

    const combined =
existing.concat(rows).slice(-getFrameworkTimingRetentionLimit_());

    // Add skipShellCheck to stop redundant verification when
writing Section D

    replaceFrameworkTimingSectionRows_(sheet, "D", combined, {
skipStyle: true, skipShellCheck: true }

```

This is an excellent idea for maintaining the framework over time. By hashing the Format Dashboard sections and keeping a running log in the **Dashboard Quality Report**, you will immediately know exactly which internal function needs to be updated (e.g., `getDefaultColumnDefinitionRows_`) whenever a user makes a structural tweak to the dashboard UI.

To do this, we will repurpose the unused **SECTION G** into a dedicated **Format Dashboard Changelog**.

Here are the updates required to wire this tracking engine into the core validation workflow.

1. Update the Dashboard Quality Registry

Replace your existing `RFF_DASHBOARD_QUALITY_SECTIONS` and add the new `RFF_DASHBOARD_CHANGELOG_KEY` constant near the top of your Dashboard Quality code:

JavaScript

None

```
const RFF_DASHBOARD_CHANGELOG_KEY = "Format Dashboard Changelog";

const RFF_DASHBOARD_QUALITY_SECTIONS = [

  { key: RFF_DASHBOARD_VERIFY_GLOBAL_KEY, title: "SECTION A - GLOBAL INPUTS VERIFICATION" },

  { key: RFF_DASHBOARD_VERIFY_SHEETS_KEY, title: "SECTION B - SHEET DEFINITIONS VERIFICATION" },

  { key: RFF_DASHBOARD_VERIFY_HEADERS_KEY, title: "SECTION C - SHEET HEADERS VERIFICATION" },

  { key: RFF_DASHBOARD_VERIFY_COLUMNS_KEY, title: "SECTION D - COLUMN DEFINITIONS VERIFICATION" },

  { key: RFF_DASHBOARD_VERIFY_BEHAVIORS_KEY, title: "SECTION E - SHEET BEHAVIOR VERIFICATION" },

  { key: RFF_VALIDATION_SHEET, title: "SECTION F - TEMPLATE STRUCTURE & VALIDATION" },

  { key: RFF_DASHBOARD_CHANGELOG_KEY, title: "SECTION G - FORMAT DASHBOARD CHANGELOG" },

  { key: RFF_HEALTH_CHECK_SHEET, title: "SECTION H - FRAMEWORK HEALTH CHECK" },

  { key: RFF_PERFORMANCE_SUMMARY_KEY, title: "SECTION I - PERFORMANCE SUMMARY" },

  { key: RFF_MASTER_LIST_HEALTH_KEY, title: "SECTION J - MASTER LIST VALIDATION" },
```

```

    { key: RFF_CP_SYNC_DIAGNOSTICS_KEY, title: "SECTION K - CARE PLAN SYNC VALIDATION" },

    { key: RFF_WORKFLOW_SYNC_VERIFICATION_KEY, title: "SECTION L - WORKFLOW & SYNCHRONIZATION VERIFICATION" },

    { key: "Summary", title: "SECTION M - SUMMARY" },

    { key: "Signoff", title: "SECTION N - SIGNOFF" }

];

```

2. Add the Hashing and Tracking Engine

Add these two new functions anywhere in your Dashboard Quality code. They read the Format Dashboard, hash the active data, compare it to the previously saved run, and translate the section name directly to the exact JavaScript function you need to update.

JavaScript

None

```

function getScriptFunctionForDashboardSection_(sectionTitle) {

    if (sectionTitle.indexOf("SECTION A") !== -1) return
    "getDefaultGlobalSettingsRows_()";

    if (sectionTitle.indexOf("SECTION B") !== -1) return
    "getDefaultTitleRowRows_()";

    if (sectionTitle.indexOf("SECTION C") !== -1) return
    "getDefaultSheetDefinitionRows_()";

    if (sectionTitle.indexOf("SECTION D") !== -1) return
    "getDefaultColumnDefinitionRows_()";

    if (sectionTitle.indexOf("SECTION E") !== -1) return
    "getDefaultBehaviorRows_()";

    if (sectionTitle.indexOf("SECTION F") !== -1) return
    "getDefaultSheetHeaderRows_()";
}

```



```
    if (sectionTitle.indexOf("SECTION G") !== -1) return  
    "getDefaultSystemSurfaceRows_()";
```

```
    return "Script Defaults";  
}
```

```
function updateFormatDashboardChangelog_(dashboardSheet, timing) {
```

```
    const props = PropertiesService.getDocumentProperties();
```

```
    let logRows =  
    getDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY);
```

```
    if (!logRows || logRows.length === 0) {  
        logRows = [ ["Timestamp", "Section Changed", "Event", "Action  
Required", "New Hash", "", ""] ];  
    }
```

```
    const sectionsToTrack = [  
        RFF_SECTION_GLOBAL, RFF_SECTION_TITLE_ROWS, RFF_SECTION_SHEETS,  
        RFF_SECTION_COLUMNS, RFF_SECTION_BEHAVIORS,  
        RFF_SECTION_HEADERS, RFF_SECTION_SYSTEM_SURFACES  
    ];
```

```
    let changed = false;
```

```
    const now = new Date();
```

```
sectionsToTrack.forEach(function(sectionTitle) {  
    try {  
        const rows = readDashboardSectionRows_(dashboardSheet,  
sectionTitle);  
  
        const currentHash = computeStableHash_(JSON.stringify(rows));  
  
        const propKey = RFF_QA_SECTION_PROP_PREFIX + "TRACKER_HASH_"  
+ computeStableHash_(sectionTitle);  
  
        const previousHash = props.getProperty(propKey);  
  
        // If a previous hash exists and differs, log the exact  
function requiring an update  
  
        if (previousHash && previousHash !== currentHash) {  
  
            const scriptFn =  
getScriptFunctionForDashboardSection_(sectionTitle);  
  
            logRows.splice(1, 0, [  
  
                now,  
  
                sectionTitle,  
  
                "MODIFIED",  
  
                "Update " + scriptFn + " to make permanent",  
  
                currentHash,  
  
                "",  
  
                ""  
            ]  
        )  
    }  
}
```

```

    });

    changed = true;
  }

  props.setProperty(propKey, currentHash);
} catch (err) {
  // Missing sections are skipped safely
}
});

if (changed) {
  // Retain header + top 50 changes to prevent bloat
  const trimmedLog = [logRows[0]].concat(logRows.slice(1, 51));
  saveDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY,
  trimmedLog, { deferSheetWrite: true });

  if (timing) markFrameworkStep_(timing, "Format Dashboard
  Changelog updated with modifications");
}

return changed;
}

```

3. Wire the Changelog into the Verification Workflow

Update `runDashboardQualityDashboardVerificationSections_` by replacing the existing function with this version. This ensures the changelog is evaluated every time the dashboard is verified.

JavaScript

None

```
function runDashboardQualityDashboardVerificationSections_(timing,
qualitySheet) {

    const ss = SpreadsheetApp.getActiveSpreadsheet();

    const quality = qualitySheet ||
ensureDashboardQualityReportSheet_();

    const dashboard = ss.getSheetByName(RFF_DASHBOARD_SHEET);

    if (!dashboard) {

        const missingRows = [

            ["Verification Area", "Verification Item", "Expected Value",
"Actual Value", "Status", "Severity", "Quality Notes"],

            ["Format Dashboard", "Sheet Exists", RFF_DASHBOARD_SHEET,
"Missing", "FAIL", "Critical", "Create or rebuild Format Dashboard
before template creation."],

        ];

        saveDashboardQualitySectionRows_(RFF_DASHBOARD_VERIFY_GLOBAL_KEY,
missingRows, { deferSheetWrite: true });

        replaceDashboardQualitySectionRows_(

            quality,
```

```

getDashboardQualitySectionTitleForKey_(RFF_DASHBOARD_VERIFY_GLOBAL_
KEY),

buildTimestampedDashboardQualitySectionRows_(RFF_DASHBOARD_VERIFY_G
LOBAL_KEY, missingRows)

    );

    markFrameworkStep_(timing, "Format Dashboard missing");

    return missingRows;
}

const dashboardIndex = buildDashboardSectionIndex_(dashboard);

// --- RUN CHANGELOG TRACKER ---

const changelogUpdated =
updateFormatDashboardChangelog_(dashboard, timing);

const sectionPayloads = [

    { key: RFF_DASHBOARD_VERIFY_GLOBAL_KEY, rows:
collectFormatDashboardGlobalInputVerificationRows_(dashboardIndex),
label: "Section A Global Inputs and Section B Title Rows verified"
},

    { key: RFF_DASHBOARD_VERIFY_SHEETS_KEY, rows:
collectFormatDashboardSheetDefinitionVerificationRows_(dashboardInd
ex), label: "Section C Sheet Definitions verified" },

```

```

    { key: RFF_DASHBOARD_VERIFY_HEADERS_KEY, rows:
collectFormatDashboardSheetHeaderVerificationRows_(dashboardIndex),
label: "Section F Sheet Headers verified" },

    { key: RFF_DASHBOARD_VERIFY_COLUMNS_KEY, rows:
collectFormatDashboardColumnDefinitionVerificationRows_(dashboardIndex),
label: "Section D Column Definitions verified" },

    { key: RFF_DASHBOARD_VERIFY_BEHAVIORS_KEY, rows:
collectFormatDashboardSheetBehaviorVerificationRows_(dashboardIndex),
label: "Section E Sheet Behaviors verified" },

    { key: RFF_DASHBOARD_CHANGELOG_KEY, rows:
getDashboardQualitySectionRows_(RFF_DASHBOARD_CHANGELOG_KEY),
label: "Section G Format Dashboard Changelog verified" }

];

```

```

const sectionWrites = [];

sectionPayloads.forEach(function(item) {

    const previousRows = getDashboardQualitySectionRows_(item.key);

    const changed = !dashboardQualityRowsEqualValues_(previousRows,
item.rows) || (item.key === RFF_DASHBOARD_CHANGELOG_KEY &&
changelogUpdated);

    saveDashboardQualitySectionRows_(item.key, item.rows, {
deferSheetWrite: true });

    markFrameworkStep_(timing, item.label + (changed ? "" : "
skipped - unchanged"));

    if (changed) {

```

```

        sectionWrites.push({
            title: getDashboardQualitySectionTitleForKey_(item.key),
            rows:
buildTimestampedDashboardQualitySectionRows_(item.key, item.rows)
        });
    }
});

if (sectionWrites.length) {
    replaceDashboardQualitySectionsRows_(quality, sectionWrites);
    markFrameworkStep_(timing, "Dashboard Quality Sections A-G
batch written");
} else {
    markFrameworkStep_(timing, "Dashboard Quality Sections A-G
sheet write skipped - unchanged");
}

return true;
}

```

4. Apply the Formatting Adjustments

Replace `getDefaultDashboardQualityDetailHeader_` and `normalizeDashboardQualityRowsForSection_` with these updated versions to format Section G correctly.

JavaScript

None

```
function getDefaultDashboardQualityDetailHeader_(section) {

    const title = String(section && section.title || "");

    if (title.indexOf("SECTION A - ") === 0) return ["Setting",
    "Status", "Issue", "Quality Notes", "", "", ""];

    if (title.indexOf("SECTION B - ") === 0) return ["Sheet",
    "Status", "Issue", "Quality Notes", "", "", ""];

    if (title.indexOf("SECTION C - ") === 0) return ["Header",
    "Status", "Issue", "Quality Notes", "", "", ""];

    if (title.indexOf("SECTION D - ") === 0) return ["Column",
    "Status", "Issue", "Quality Notes", "", "", ""];

    if (title.indexOf("SECTION E - ") === 0) return ["Behavior",
    "Status", "Issue", "Quality Notes", "", "", ""];

    if (title.indexOf("SECTION F - ") === 0) return ["Template",
    "Status", "Issue", "Quality Notes", "", "", ""];

    // Section G - Changelog Headers

    if (title.indexOf("SECTION G - ") === 0) return ["Timestamp",
    "Section Changed", "Event", "Action Required", "New Hash", "", ""];

    if (title.indexOf("SECTION H - ") === 0) return ["Area", "Item",
    "Status", "Quality Notes", "", "", ""];

    if (title.indexOf("PERFORMANCE SUMMARY") !== -1) return
    ["Process", "Runtime", "Status", "Quality Notes", "", "", ""];
```



```

    if (title.indexOf("MASTER LIST VALIDATION") !== -1) return
["Validation Item", "Status", "Issue", "Quality Notes", "", "",
""];

    if (title.indexOf("CARE PLAN SYNC VALIDATION") !== -1) return
["Diagnostic", "Status", "Issue", "Quality Notes", "", "", ""];

    if (title.indexOf("WORKFLOW & SYNCHRONIZATION VERIFICATION") !==
-1) return ["Sync Check", "Status", "Issue", "Quality Notes", "",
"", ""];

    if (title.indexOf("SUMMARY") !== -1) return ["Metric", "Value",
"Status", "Quality Notes", "", "", ""];

    if (title.indexOf("SIGNOFF") !== -1) return ["Audit Item",
"Status", "Issue", "Quality Notes", "", "", ""];

    return ["Item", "Status", "Issue", "Quality Notes", "", "", ""];
}

```

```

function normalizeDashboardQualityRowsForSection_(section, rows) {

    const input = rows || [];

    const governedHeader =
getDefaultDashboardQualityDetailHeader_(section);

    if (!input.length) return [governedHeader,
normalizeDashboardQualityOutputRow_(["Enter Data here"], null)];

    const letter = getDashboardQualitySectionLetter_(section);

    const output = [governedHeader];

```

```

for (let r = 1; r < input.length; r++) {

    const row = input[r] || [];

    let mapped;

    if (letter === "A") mapped = [row[1] || row[0] || "", row[4] ||
row[1] || "", normalizeDashboardQualityIssueValue_(row[3], row[2]),
row[6] || row[3] || ""];

    else if (letter === "B") mapped = [row[1] || row[0] || "",
row[4] || row[1] || "",
normalizeDashboardQualityIssueValue_(row[3], row[2]), row[6] ||
row[3] || ""];

    else if (letter === "C") mapped = [row[1] ? String(row[0] ||
"") + " - " + String(row[1]) : row[0] || "", row[4] || row[1] ||
"", normalizeDashboardQualityIssueValue_(row[3], row[2]), row[6] ||
row[3] || ""];

    else if (letter === "D") mapped = [row[1] ? String(row[0] ||
"") + " - " + String(row[1]) : row[0] || "", row[4] || row[1] ||
"", normalizeDashboardQualityIssueValue_(row[3], row[2]), row[6] ||
row[3] || ""];

    else if (letter === "E") mapped = [row[1] ? String(row[0] ||
"") + " - " + String(row[1]) : row[0] || "", row[4] || row[1] ||
"", normalizeDashboardQualityIssueValue_(row[3], row[2]), row[6] ||
row[3] || ""];

    else if (letter === "F") mapped = [row[0] || "", row[2] ||
row[1] || "", row[3] || row[4] || "", row[6] || row[3] || ""];

    // Section G Normalization (Preserves Timestamp and Action
Required column layout)

```

```
else if (letter === "G") mapped = [
    row[0] instanceof Date ? row[0] : (row[0] || ""),
    row[1] || "",
    row[2] || "",
    row[3] || "",
    row[4] || "",
    row[5] || "",
    row[6] || ""
];

else if (letter === "H") mapped = [row[0] || "", row[1] || "",
row[2] || "", row[3] || ""];

else if (letter === "I") mapped = [row[0] || "", row[2] !==
undefined ? row[2] : "", row[4] || "", row[6] || ""];

else if (letter === "J") mapped = [row[0] || "", row[1] || "",
row[2] || "", row[3] || ""];

else if (letter === "K") mapped = [row[0] || "", row[1] || "",
row[2] || "", row[3] || ""];

else if (letter === "L") mapped = [row[0] || "", row[2] ||
row[1] || "", row[4] || row[2] || "", row[6] || row[3] || ""];

else if (letter === "M" || letter === "N" || letter === "O")
mapped = [row[0] || "", row[1] || "", row[2] || "", row[3] || ""];

else mapped = [row[0] || "", row[1] || "", row[2] || "", row[3]
|| ""];
```

```

        output.push(normalizeDashboardQualityOutputRow_(mapped, null));
    }

    if (output.length === 1)
    output.push(normalizeDashboardQualityOutputRow_(["Enter Data
here"], null));

    return output;
}

```

Here is the complete implementation for all your requested updates. I have broken them down by their specific architectural areas so you can easily drop them into your codebase.

1. Format Dashboard Section Reordering

First, we need to update the constants at the top of your script to reflect the new letter assignments, and then update the fast-writer to output them in the correct visual order.

Update your constants:

JavaScript

```

None
const RFF_SECTION_GLOBAL = "SECTION A - GLOBAL SETTINGS";

const RFF_SECTION_TITLE_ROWS = "SECTION B - TITLE ROWS";

const RFF_SECTION_SHEETS = "SECTION C - SHEET DEFINITIONS";

const RFF_SECTION_BEHAVIORS = "SECTION D - SHEET BEHAVIORS";

const RFF_SECTION_SYSTEM_SURFACES = "SECTION E - SYSTEM SHEET
SURFACES";

const RFF_SECTION_COLUMNS = "SECTION F - COLUMN DEFINITIONS";

```

```
const RFF_SECTION_HEADERS = "SECTION G - SHEET HEADERS";
```

Update `writeDashboardDefaultsFast_`:

Replace your existing function with this to output the sections in the new order:

JavaScript

None

```
function writeDashboardDefaultsFast_(sheet) {

    const width = 14;

    const rows = [];

    rows.push(normalizeSectionRowForWidth_([ "Format Dashboard", "- v"
+ RFF_VERSION + " -", "Report Formatter Framework"], width));

    rows.push(normalizeSectionRowForWidth_([], width));


    appendDashboardSectionRows_(rows, RFF_SECTION_GLOBAL, ["Setting",
"Value", "Options"], getDefaultGlobalSettingsRows_(), width);

    appendDashboardSectionRows_(rows, RFF_SECTION_TITLE_ROWS, ["Sheet
Type", "Row", "Purpose", "Value Source", "Label", "Target Cell",
"Height", "Font Size", "Font Weight", "Fill Level", "Alignment",
"Wrap", "Notes"], getDefaultTitleRowRows_(), width);

    appendDashboardSectionRows_(rows, RFF_SECTION_SHEETS, ["Sheet
Type", "Report Title", "Template Name", "Output Naming Pattern",
"Base Color", "Use Prompt Date", "End Date Source", "Template Row
Count", "Template Column Count", "Template Row Mode", "Minimum
Rows", "Buffer Rows", "Test Rows", "Use Test Rows"],
getDefaultSheetDefinitionRowsWithColumnCounts_(), width);


    // NEW ORDER: Behaviors -> System Surfaces -> Columns -> Headers
```

```
    appendDashboardSectionRows_(rows, RFF_SECTION_BEHAVIORS, ["Sheet  
Type", "Uses Title Rows", "Uses Filter", "Uses Alternating Colors",  
"Uses Subheaders", "Hidden Template", "Output Visibility"],  
getDefaultBehaviorRows_(), width);
```

```
    appendDashboardSectionRows_(rows, RFF_SECTION_SYSTEM_SURFACES,  
["System Sheet Name", "Display Name", "Sort Order", "Output  
Visibility", "Title Fill Color", "Title Font Color", "Use Global  
Defaults", "Notes"], getDefaultSystemSurfaceRows_(), width);
```

```
    appendDashboardSectionRows_(rows, RFF_SECTION_COLUMNS, ["Header",  
"Width", "Header Font Size", "Date Column", "Hide Column", "Data  
Wrap", "Horizontal Alignment", "Vertical Alignment", "Number  
Format"], getResolvedDefaultColumnDefinitionRows_(), width);
```

```
    appendDashboardSectionRows_(rows, RFF_SECTION_HEADERS, ["Sheet  
Type", "Column Order", "Header", "Source of Data"],  
getDefaultSheetHeaderRows_(), width);
```

```
    trimSheetToColumnCount_(sheet, width, "Format Dashboard");
```

```
    if (sheet.getMaxRows() < rows.length)  
sheet.insertRowsAfter(sheet.getMaxRows(), rows.length -  
sheet.getMaxRows());
```

```
    const clearRows = Math.max(sheet.getLastRow(), rows.length, 1);
```

```
    sheet.getRange(1, 1, clearRows,  
width).clearContent().breakApart().setBackground("#FFFFFF");
```

```
    sheet.getRange(1, 1, rows.length, width).setValues(rows);
```

```
    sheet.getRange(1, 1, rows.length, width)
```

```
.setFontFamily("Arial")

.setFontColor("#000000")

.setFontSize(10)

.setHorizontalAlignment("left")

.setVerticalAlignment("middle")

.setWrapStrategy(SpreadsheetApp.WrapStrategy.CLIP);

sheet.getRange(1, 1, 1, width)

    .setFontSize(16)

    .setFontWeight("bold")

    .setBackground("#D9EAF7");

const sectionRanges = [];

const sectionHeaderRanges = [];

for (let r = 0; r < rows.length; r++) {

    const first = String(rows[r][0] || "").trim();

    if (first.indexOf("SECTION ") === 0) {

        sectionRanges.push(rowColToA1_(r + 1, 1) + ":" +
rowColToA1_(r + 1, width));

        if (r + 2 <= rows.length) {

            sectionHeaderRanges.push(rowColToA1_(r + 2, 1) + ":" +
rowColToA1_(r + 2, width));
```

```

        }
    }
}

if (sectionRanges.length) {
    sheet.getRangeList(sectionRanges)
        .setFontSize(12)
        .setFontWeight("bold")
        .setBackground("#B7DEE8");
}

if (sectionHeaderRanges.length) {
    sheet.getRangeList(sectionHeaderRanges)
        .setFontWeight("bold")
        .setBackground("#D9EAF7")
        .setWrapStrategy(SpreadsheetApp.WrapStrategy.WRAP);
}

// Set Top rows to overflow

sheet.getRange(1, 1, 3,
width).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

const widths = [250, 160, 240, 180, 130, 120, 160, 150, 140, 120,
120, 120, 120];

```



```
widths.forEach(function(colWidth, index) {  
    try {  
        sheet.setColumnWidth(index + 1, colWidth);  
    } catch (err) { }  
});  
  
sheet.setFrozenRows(1);  
}  
  
// Update the changelog parser to match the new letters  
function getScriptFunctionForDashboardSection_(sectionTitle) {  
    if (sectionTitle.indexOf("SECTION A") !== -1) return  
    "getDefaultGlobalSettingsRows_()";  
  
    if (sectionTitle.indexOf("SECTION B") !== -1) return  
    "getDefaultTitleRowRows_()";  
  
    if (sectionTitle.indexOf("SECTION C") !== -1) return  
    "getDefaultSheetDefinitionRows_()";  
  
    if (sectionTitle.indexOf("SECTION D") !== -1) return  
    "getDefaultBehaviorRows_()";  
  
    if (sectionTitle.indexOf("SECTION E") !== -1) return  
    "getDefaultSystemSurfaceRows_()";  
  
    if (sectionTitle.indexOf("SECTION F") !== -1) return  
    "getDefaultColumnDefinitionRows_()";  
}
```

```

    if (sectionTitle.indexOf("SECTION G") !== -1) return
    "getDefaultSheetHeaderRows_()";

    return "Script Defaults";

}

```

2. Disenrolled Exclusion & Demo P Dates (Last Day / Last Updated)

For Disenrolled Exclusion: Replace `updateDisenrolledExclusionReportDates_` to ensure D2 is strictly the last day of the month, and inject the timestamp in E2.

JavaScript

None

```

function updateDisenrolledExclusionReportDates_(sheet, monthParts)
{
    if (!sheet) return;

    const startDate = monthParts && monthParts.firstDay ?
monthParts.firstDay : getTodayLocalDate_();

    const endDate = monthParts && monthParts.lastDay ?
monthParts.lastDay : startDate;

    sheet.getRange("A1").setValue(DISENROLLED_EXCLUSION_SHEET);

    sheet.getRange("A2").setValue("Date:");

    sheet.getRange("B2").setValue(startDate);

    sheet.getRange("C2").setValue("to");

    sheet.getRange("D2").setValue(endDate); // Explicitly the last
day of the month

```

```

    // Add Last Updated

    const timestamp = Utilities.formatDate(new Date(),
Session.getScriptTimeZone(), "MM/dd/yyyy hh:mm a");

    sheet.getRange("E2").setValue("Last Updated: " +
timestamp).setFontStyle("italic");

}

```

For Demo P: Update the bottom of `createActiveDemoPFromRawData_` (and `updateExistingDemoPFromRawData_` if you use it) to add the E2 stamp.

JavaScript

None

```

    // Inside createActiveDemoPFromRawData_ (Near the end, after
applying title values)

    try {

        demoSheet.getRange("A1").setValue(sheetDef.reportTitle || "Demo
P");

        if (monthParts) {

            demoSheet.getRange("B2").setValue(monthParts.firstDay);

            demoSheet.getRange("D2").setValue(monthParts.lastDay);

        }

        // Add Last Updated

        const timestamp = Utilities.formatDate(new Date(),
Session.getScriptTimeZone(), "MM/dd/yyyy hh:mm a");

```

```

    demoSheet.getRange("E2").setValue("Last Updated: " +
timestamp).setFontStyle("italic");

    const existingFilter = demoSheet.getFilter &&
demoSheet.getFilter();

    if (existingFilter) existingFilter.remove();

    demoSheet.getRange(HEADER_ROW, 1,
Math.max(demoSheet.getMaxRows() - HEADER_ROW + 1, 1),
width).createFilter();

} catch (err) {

    logBestEffortWarning_("Demo P metadata update skipped: " +
err.message);

}

```

3. Demo P "Update Month" Format Bug

Because Sheets interprets raw text like "06.26" as a serial number or math function, we must pass the actual `Date` object so the format governance engine handles it correctly.

Replace `populateDemoPUpdateColumns_`:

JavaScript

None

```

function populateDemoPUpdateColumns_(data, monthParts,
sourceSheetName, status) {

    if (!data || !data.headerMap || !data.values) return;

    const statusIdx = data.headerMap["Demo P Update Status"];

    const monthIdx = data.headerMap["Demo P Update Month"];

    const sourceIdx = data.headerMap["Demo P Source Sheet"];

```

```

    if (statusIdx === undefined && monthIdx === undefined &&
sourceIdx === undefined) return;

    // Pass the raw Date object, NOT the formatted "MM.yy" text
string

    const monthValue = monthParts && monthParts.firstDay ?
monthParts.firstDay : "";

    data.values.forEach(function(row) {

        if (statusIdx !== undefined) row[statusIdx] = status ||
"Created";

        if (monthIdx !== undefined) row[monthIdx] = monthValue;

        if (sourceIdx !== undefined) row[sourceIdx] = sourceSheetName
|| "";

    });

}

```

4. Missing Monthly Change Report Warning

Update the top of `updateDemoPMonthlySyncForMonth_` to enforce the existence of the Monthly Change Report before it touches any data.

JavaScript

```

None

function updateDemoPMonthlySyncForMonth_(monthParts, timing,
options) {

    if (!monthParts) return null;

    options = options || {};

```

```
    const timingPrefix = options.timingPrefix === undefined ? "Create  
Monthly Update - Update Demo P - " : String(options.timingPrefix ||  
    "");  
  
    const step = function(label) { if (timing)  
markFrameworkStep_(timing, timingPrefix + label); };  
  
    const ss = SpreadsheetApp.getActiveSpreadsheet();  
  
    // --- NEW: ABORT IF MONTHLY CHANGE IS MISSING ---  
  
    const mcrSheet = getMonthlySheetByPrefixAndDate_(ss,  
MONTHLY_CHANGE_REPORT_PREFIX, monthParts.firstDay,  
monthParts.lastDay);  
  
    if (!mcrSheet) {  
  
        const errorMsg = "Update Demo P stopped: Monthly Change Report  
for " + formatReportDateLabel_(monthParts.firstDay) + " was not  
found. Please run Build Monthly Change Report first.";  
  
        notify_(errorMsg);  
  
        throw new Error(errorMsg);  
  
    }  
  
    const demoSheet = getCurrentDemoPSheet_(monthParts);  
  
    if (!demoSheet) throw new Error("Ongoing Demo P sheet was not  
found. Run Build Demo P (Initialization) first.");  
  
    step("Locate ongoing Demo P sheet: " + demoSheet.getName());
```

```
// ... (Rest of the function remains identical) ...
```

5. System Sheets Overflow (Rows 1-3)

To ensure the top rows overflow naturally without stretching the height of the cells, update `applyDashboardQualityReportColumnSettings_` and `styleFrameworkTimingReport_`.

In `applyDashboardQualityReportColumnSettings_`:

JavaScript

None

```
// Replace this block near the bottom of
applyDashboardQualityReportColumnSettings_

try {

    sheet.getRange(1, 1, lastRow, lastCol)

        .setFontFamily("Arial")

        .setFontSize(10)

        .setVerticalAlignment("top")

        .setHorizontalAlignment("left")

        .setBorder(true, true, true, true, true, true,
globals.globalBorderColor || "#CCCCCC",
getGlobalBorderStyle_(globals));

    // Force rows 1-3 to Overflow

    sheet.getRange(1, 1, 3,
lastCol).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

    // Everything from Row 4 down wraps normally
```

```

    if (lastRow > 3) {

        sheet.getRange(4, 1, lastRow - 3,
1).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

        sheet.getRange(4, 2, lastRow - 3, Math.max(lastCol - 1,
1)).setWrapStrategy(SpreadsheetApp.WrapStrategy.WRAP);

    }

    sheet.setFrozenRows(1);

} catch (err) {

    logBestEffortWarning_("Dashboard Quality base style skipped: "
+ err.message);

}

```

In styleFrameworkTimingReport_:

JavaScript

None

```

    // Inside the styling loop of styleFrameworkTimingReport_

    sheet.getRange(item.startRow, 1, rowCount,
1).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

    // Check if we are styling the top title rows vs the data
rows

    if (item.startRow <= 3) {

        sheet.getRange(item.startRow, 1, rowCount,
width).setWrapStrategy(SpreadsheetApp.WrapStrategy.OVERFLOW);

```



```

    } else {

        [2, 3, 4, 5, 6, 7, 8].forEach(function(col) {

            sheet.getRange(item.startRow, col, rowCount,
1).setWrapStrategy(SpreadsheetApp.WrapStrategy.WRAP);

        });

    }

```

6. Group Index "Imported Data" By Month

Update `createIndexSheet()` to dynamically map the Imported Data sheets into discrete month-based categories.

JavaScript

None

```

// INSIDE createIndexSheet()

// Replace your existing addIndexSection_("Imported Data", ...)
block with this:

```

```

const importedPredicate = function(sheetName) {

    return sheetName.indexOf("Raw Data") === 0 ||

        sheetName.indexOf(BANNER_PREFIX) === 0 ||

        sheetName.indexOf(BANNER_REPORT_ALT_PREFIX) === 0 ||

        sheetName.indexOf(CARE_PLAN_DUE_PREFIX) === 0 ||

        sheetName.indexOf(CARE_PLAN_DUE_DATE_ALT_PREFIX) === 0 ||

        sheetName.indexOf(UNLOCKED_PREFIX) === 0 ||

        sheetName.indexOf("Unlocked Care Plan") === 0;

```

```

};

    const allImported = getIndexSheetNamesInMonthSortOrder_(ss,
importedPredicate);

    const importedByMonth = {};

    allImported.forEach(function(name) {

        const d = extractFirstDateFromSheetName_(name);

        // Group them by MM.yy (e.g., "05.26")

        const mLabel = d ? Utilities.formatDate(d,
Session.getScriptTimeZone(), "MM.yy") : "Unknown Month";

        if (!importedByMonth[mLabel]) importedByMonth[mLabel] = [];

        importedByMonth[mLabel].push(name);

    });

    // Sort months descending, and output a new section for each
month

    Object.keys(importedByMonth).sort(function(a, b) { return
b.localeCompare(a); }).forEach(function(mLabel) {

        addIndexSection_("Imported Data - " + mLabel,
importedByMonth[mLabel]);

    });

```